プログラミング言語実験 −第 1 回−

2412747 江尾祐哉

1 問目

・実行方法

「./2412747_kadai1-1 10 24」のようにして第一引数に鶴と亀の合計数、第二引数に鶴と亀の足の合計本数を入力。

・実装概要

まず入力エラーだが、

1. 引数が 2 つでない
2. 入力された値が負の値
3. 入力された値が小数
4. 入力されたものが数字ではない

のときに弾くようにした。2 と 3 は要件内だが、1 は実行が間違えたときに SEGV になるため行い(以降の問題でも同様)、4 は判断して入れた。

入力されたものは文字列であるため、それを int 型に変換し、その値に関してもやはりエラーチェックを行った。鶴の足の本数は 2 本であり、亀の足の本数は 4 本であることに留意すると、

1. 足の本数が奇数
2. 足の本数が、全て鶴であったと仮定したときより少ない
3. 足の本数が、全て亀であったと仮定したときより多い

を弾けば良いとわかる。

その後は計算部分となる。先ほどのエラー処理でも出た考え方だが、全て亀であると仮定して、合計数を亀の数とし、鶴は 0 匹であるとした。その後、足の合計本数が

合うまで、順番に亀の数を減らして鶴の数を増やしていく。そうすると最終的な答えが出ることになる。

なおエラー文だが、標準エラー出力をする方が親切だと感じたので、std::cout ではなく std::cerr を使用した。また、エラー時に exit(EXIT_FAILURE)を記述してマジックナンバーの使用を避けているが、0 が成功で 1 が失敗というのは人口に膾炙しているので、0 や 1 を使用するのでもよいとは感じている。

・実行結果

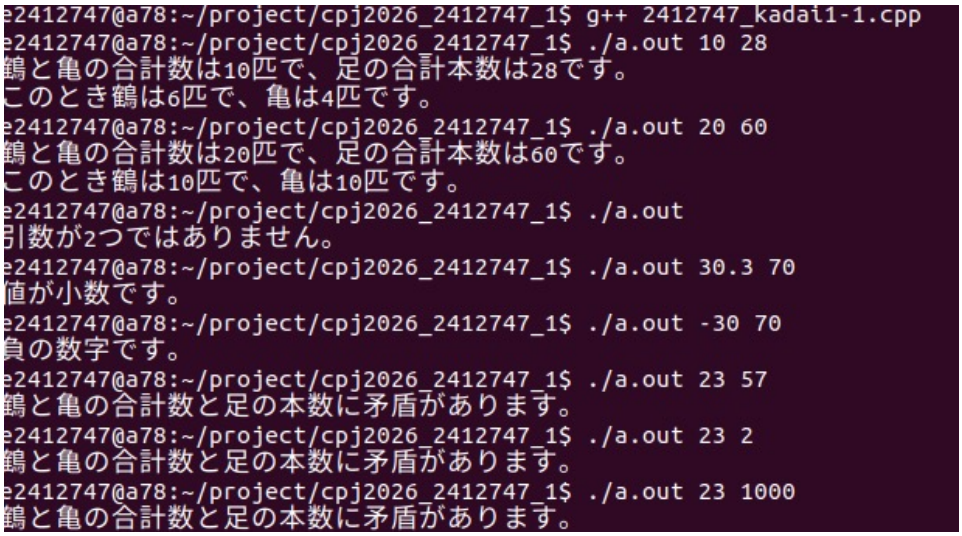

図1　課題1の実行結果

2 問目

・実行方法

コンパイルして生成された実行ファイルを実行、その後に入力ファイル名と出力ファイル名をそれぞれ標準入力する。

・実装概要

ファイルの内容があらかじめわかっている状態なので、静的に int 型の配列を確保し、そこに順々にファイル内に記載されている数字を代入していけばよい。一行につ

き、それぞれ学籍番号、数学の点数、英語の点数、国語の点数が順番に載っており、今回必要なのは各教科の点数なので、読み取りが$4n+1$、$4n+2$、$4n+3$、$4n$回でそれぞれ条件分岐し、そのうち$4n+1$は不要なので特に処理をせず、そのほかの場合は int 型で宣言した各教科の平均点の変数に代入していけば良い。また、合計点は$4n+1$回以外で毎回代入していけば良い。最後にそれぞれの変数を人数である 50 人で割ると、無事平均点が求まる。あとは std::cout と std::fout で出力する。

なお、エラーの処理はファイルの読み取りの失敗の場合のみ行っている。今回は標準入力でファイル名を指定し、引数を使わないので引数処理は必要としない。

・実行結果

```
e2412747@a78:~/project/cpj2026_2412747_1$ g++ 2412747_kadai1-2.cpp
e2412747@a78:~/project/cpj2026_2412747_1$ ./a.out
入力するファイル名は？
test2026.txt
出力するファイル名は？
result.txt
数学の平均点は54点で、英語の平均点は43点で、国語の平均点は46点です。
また、合計点の平均点は143点です。
```

図2　課題2の実行結果

3 問目

・実行方法

「./2412747_kadai1-3 sentence2026.txt」のようにして第一引数に読み取るファイル名を入力する。

・実装概要

ファイルにはアルファベットの羅列が記載されているとなっているので、ファイルの文字を読み取って行って大文字ならば小文字にして出力、小文字ならば大文字にして出力、それ以外ならそのまま出力とするだけで完了する。しかしながら、今までのように fin >> word のようにしてしまうと、空白や改行等が省略されて詰められた状態で文字

が出力されてしまう。ここがこの課題のポイントであると考えた。そのため、fin >> noskipws >> word とすることで、空白や改行を省略しない状態で読み取るように、すなわちファイルを一文字ずつそのまま読み取る形でコードを書くことで、元ファイルの状態を反映させながら大文字小文字を逆転させたものを出力できる。

・実行結果

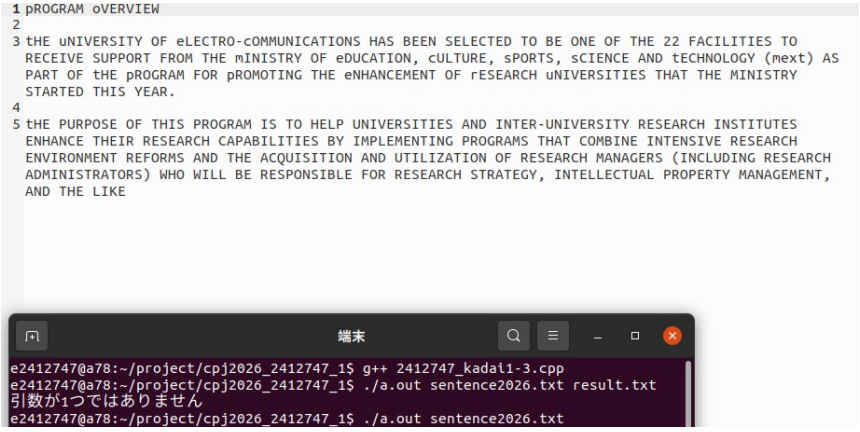

図3　課題3の実行結果

4 問目

・実行方法

「./2412747_kadai1-4 diff1_2026.txt diff2_2026.txt」のようにして第一引数と第二引数にそれぞれ差分をとるファイル名を入力する。

・実装概要

　やっていることは 2 つのファイルからそれぞれ単語を読み取り、それを比較して共通部分とそうでない部分にそれぞれ分けるということなので、課題のアルゴリズムを考えるのはそう難しくない。しかしそのままやろうとすると、入力ファイルの単語を格納する変数 2 つ、共通部分の単語を格納する変数 1 つ、共通でない単語を格納する変数 2 つ

と計 5 個の変数を必要とし、この変数を配列とすると管理が非常に大変である。さらには元ファイルには重複して出現する単語もあるため、それを考慮して重複がでないようにするとさらにコードは複雑になる。

そこで調べてみると、コンテナと呼ばれる便利なものがあった。コンテナを用いると、ファイルから読み取った単語で重複があった際に自動で排除することができ、また単語が自動でソートされることになる。また、for 文を用いて配列の要素を線形探索するよりも速くに検索することができる。また、コードがシンプルになるので可読性の高さも担保される。特に入力ファイルの単語数はわかっても、どれくらい共通があるのか等はわからないため、あらかじめ大きな配列を静的に確保するか、もしくは動的に確保するかとなるが、前者はメモリ領域を無駄に使うことになり、後者はコードが煩雑になる上、処理が重くなる。これらの点から今回はコンテナを利用してプログラムを作成した。

コード全体の流れとしては、まずは引数やファイル読み取りのエラーの処理をし、その後にファイルから単語を読み取っていくが、この時にそれぞれのコンテナに代入する際に、normalize 関数を用いて大文字を小文字に、文末の記号を削除という処理を行い正規化した。課題文ではカンマ、コロン、セミコロンを削除とあったが、そのほかにもピリオドも削除するべきだと考えたので、追加した。読み取りが終わったら、それぞれの単語を比較していき、共通部分と diff1 単独とに分類した。そして diff2 の方も for 文の処理をし、共通部分はすでに探索したので、そうでないものを diff2 単独として分類した。あとはそれぞれの単語と単語数をファイルと標準出力それぞれに出力すれば終了である。

・実行結果

```
e2412747@a78:~/project/cpj2026_2412747$ ./a.out diff1_2026.txt diff2_2026.txt
2つのファイルに共通して出現する単語(単語数:99個)
a about after age also an and application apply are assessing authentication bec
ome but department design designing developing divided education encryption engi
neering enhancement for from fundamental gives golden how humans in include incl
udes informatics information innovators into involves knowledge later learning m
anage management managing materials methods multimedia need networking networks
not of only operating other our practical processing program programs realizatio
n receive related research resulted safety security separate service skills soci
ety students subjects support system system-management taught teaching technique
s technology that the their they third this threats three to trained use variety
 various where whose wide will work year

diff1_2026.txtのみに記載されている単語(単語数:0個)


diff2_2026.txtのみに記載されている単語(単語数:21個)
abundant aim applying as convenient delivering development engages foundation go
als graduate learn learning/teaching media please policy refer safe school using
 with
```

図4　課題4の実行結果

5 問目

・実行方法

「./2412747_kadai1-4 enrollment2026.txt input2026.txt」のようにして第一引数に各個人の虹彩領域の情報の載っているファイル名を、第二引数に特定の人物の虹彩領域の情報の載っているファイル名を入力する。

・実装概要

虹彩情報やハミング距離と初見の単語が多く、一見すると難しく感じたが、実際にやることは読み取ったビット列をそれぞれ比較して、その中で最も誤差の少ないものを出すということだけであり、単純である。

コードの流れとしては、あらかじめ虹彩情報が記載されている数がわかっているので、その分の id と各 id における虹彩情報を記録する配列を宣言することから始まる。ここで数をベタうちするのではなく、マクロを定義して PEOPLE_NUM や BIT_LEN とした方が、可読性が良いことに気がついた(思い出した)ので、その定義を用いて id を int 型の配列に、虹彩情報を id に対応させる形で char 型の二次元配列にした。なお、iris は char 型であるため、安全のために最後の 1 文字に終端文字を代入するようにした。そうしたら調べたい情報の載っているファイルからも虹彩情報を読み取り、それを順番に比較していく。比較する関数は calc_hamming_distance 関数で、これは単に配列の要素を一つずつ比較していって違っていたら、int 型の変数の dist をインクリメントしていくというものである。これを用いて最終的に最もこの値が小さくなった id と最小距離を求めれば、課題は完了となる。なお、closest_dist に BIT_LEN + 1 や closest_id に-1 を代入しているのは、センチネル値といって使用していない値を代入することで、実行部分に影響を与えないことができる上、エラーが発生したとき(今回はないが)にエラーの処理がしやすいことが利点であるので、このような工夫をした。

・実行結果

```
e2412747@a78:~/project/cpj2026_2412747_1$ g++ 2412747_kadai1-5.cpp
e2412747@a78:~/project/cpj2026_2412747_1$ ./a.out enrollment2026.txt input2026.t
xt
最も似ている人物のIDは90で、
ハミング距離は944となります。
```

図5　課題5の実行結果